Practical Assignment 2: Organizational IT Infrastructure

Gestió de Sistemes i Xarxes

Deadline: 15 May 2026

Abstract

In this assignment, you will work in pairs to design and implement a modern IT infrastructure for a growing startup. Building on foundational single-server skills from Assignment 1, you will learn containerization, orchestration, infrastructure automation, network design, and observability. Rather than mastering each technology deeply, you will develop a practical understanding of what each tool does, why it matters, and when to use it.

This is how modern infrastructure is actually built.

Overview

Duration

6 weeks (Weeks 8–13)

- **Start Date:** 7 April 2026
- **Deadline:** 15 May 2026
- **Oral Interview:** Week 14-15 (18–29 May 2026)

Format

- **Group work:** Pairs of two students
- **Weekly lab sessions:** 2 hours per week with professor
- **Autonomous work:** Most development happens outside lab
- **Tool:** Version control (Git) to track all progress

Learning Outcomes

By completing this assignment, you will demonstrate the ability to:

1. Containerize applications using Docker and understand container concepts
2. Coordinate multi-container applications with Docker Compose
3. Deploy and manage applications in Kubernetes (basic concepts)
4. Understand Infrastructure as Code and implement basic automation
5. Design organizational networks with security principles
6. Understand identity and centralized authentication concepts
7. Implement basic monitoring and observability
8. Document infrastructure and operational procedures

9. Reason about technology trade-offs and architectural choices
10. Collaborate effectively on infrastructure projects

Contents

1 The Scenario: "GreenDevCorp's Digital Transformation" 5

2 How This Assignment Works 6
2.1 Philosophy . . . 6
2.2 Challenge-Based Learning . . . 6
2.3 Difficulty Levels . . . 6
2.4 Lab Sessions . . . 6

3 Assessment 7
3.1 Process (40%) . . . 7
3.2 Functionality (30%) . . . 7
3.3 Communication (30%) . . . 7

4 Week 8: Containerization (Docker) 8
4.1 Core (* Basic): Containerize Two Applications . . . 8
4.2 Intermediate (**): Optimize Your Dockerfiles . . . 9
4.3 Advanced (***): Security Hardening . . . 9

5 Week 9: Multi-Container Orchestration (Docker Compose) 11
5.1 Core (* Basic): Create a docker-compose.yml . . . 11
5.2 Intermediate (**): Add Health Checks and Dependencies . . . 12
5.3 Advanced (***): Custom Networks and Logging . . . 12

6 Week 10: Orchestration (Kubernetes) 14
6.1 Core (* Basic): Deploy to Kubernetes (Minikube) . . . 14
6.2 Intermediate (**): Add Resource Limits and Probes . . . 15
6.3 Advanced (***): Persistent Storage . . . 15

7 Week 11: Infrastructure as Code & CI/CD 17
7.1 Core (* Basic): Write IaC and Deploy Automatically . . . 17
7.2 Intermediate (**): Multiple Environments . . . 19
7.3 Advanced (***): Advanced CI/CD . . . 19

8 Week 12: Network Design & Identity 21
8.1 Core (* Basic): Design Network Architecture . . . 21
8.2 Intermediate (**): Advanced Network Design . . . 23
8.3 Advanced (***): Identity Implementation (Optional) . . . 23

9 Week 13: Integration, Observability & Finalization 24
9.1 Challenge A: Observability with Prometheus & Grafana (Optional but Recommended) . . . 24
9.2 Challenge B: Full Integration Test (Required) . . . 25
9.3 Challenge C: Documentation (Required) . . . 25
9.4 Challenge D: Reflection & Interview Prep (Required) . . . 26

10 Grading and Assessment 28
10.1 Three Pillars . . . 28
10.2 Difficulty Levels and Expectations . . . 28
10.3 Oral Interview . . . 28

11 Guidelines and Expectations 29
11.1 Collaboration . . . 29
11.2 Problem-Solving Approach . . . 29
11.3 Version Control Best Practices . . . 29
11.4 Safety and Best Practices . . . 29
11.5 Getting Help . . . 29

12 Resources and References **30**
12.1 Recommended Reading 30
12.2 Tools and Accounts You'll Need 30
12.3 Key Commands You'll Use 30

13 Final Thoughts **31**

1 The Scenario: "GreenDevCorp's Digital Transformation"

The Company

GreenDevCorp has grown from 4 developers to 20+ people across multiple teams. The infrastructure that worked for a startup no longer scales:

- Single server with manual deployments
- Inconsistent environments (works on my laptop, why not in production?)
- No automation (deployments are manual and error-prone)
- Limited visibility (if something breaks, nobody knows until users complain)
- No growth strategy (how do we scale to 100 people?)

Your Mission

As infrastructure engineers, you will design and implement a modern system that:

- Runs applications consistently across environments
- Scales automatically to handle growth
- Recovers gracefully from failures
- Is automated and version-controlled
- Provides visibility into system behavior
- Follows industry best practices

What You'll Build

Over 6 weeks, you will:

1. **Week 8:** Package applications in containers (Docker)
2. **Week 9:** Coordinate multiple containers (Docker Compose)
3. **Week 10:** Deploy to production orchestration (Kubernetes)
4. **Week 11:** Automate infrastructure with code (IaC + CI/CD)
5. **Week 12:** Design organizational network and identity
6. **Week 13:** Integrate everything, add observability, document

By the end, you'll have a fully functional, documented infrastructure that demonstrates modern DevOps practices.

2 How This Assignment Works

2.1 Philosophy

Each week presents a new challenge that reflects real operational needs. Your job is to **discover solutions**, document your reasoning, and demonstrate your system to the professor. Unlike Assignment 1, this assignment presents **one main task per week** with optional extensions (see the difficulty levels). This approach allows you to focus on one topic at a time while being flexible about your pace and depth of engagement. The assignment emphasizes **understanding what tools do and when to use them**, rather than mastering every detail.

2.2 Challenge-Based Learning

Each week presents a real problem and asks you to solve it:

- **Challenge:** "How do we deploy consistently?"
- **Your Job:** Research, design, implement, explain
- **Scaffolding:** Use templates for syntax-heavy topics
- **Discovery:** You make architectural decisions and reason through trade-offs

2.3 Difficulty Levels

Each week offers three difficulty tiers so you can choose your level:

- *** Basic:** Core deliverable; minimum required for all students
- **** Intermediate:** Extends the core; deeper understanding
- ***** Advanced:** Optional depth; for students with time and interest

You can start at basic and upgrade to intermediate/advanced if time permits.

2.4 Lab Sessions

The weekly 2-hour lab sessions will be used to:

- **Checkpoint:** Show progress; ask questions
- **Challenge Overview:** Understand the week's problem
- **Working Time:** Build your solution; professor circulates
- **Reflection:** Plan next steps

Lab is not a lecture. It's a place to ask questions, get unblocked, and see your classmates' approaches.

3 Assessment

Your work will be evaluated on three dimensions:

3.1 Process (40%)

How well do you apply modern infrastructure practices?

- **Code Quality:** Dockerfiles, manifests, IaC code are clean and well-organized
- **Version Control:** Git history is clear; commits have meaningful messages
- **Automation:** Infrastructure is version-controlled and reproducible
- **Documentation:** Design decisions are explained; others can understand your choices
- **Collaboration:** Both team members contribute; evidence in Git history

3.2 Functionality (30%)

Does your system actually work?

- **Deployment:** Containers build and run correctly
- **Orchestration:** Kubernetes deployments work; services communicate
- **Automation:** IaC recreates infrastructure; CI validates and produces deployable artifacts
- **Networking:** Services reach each other; network policies work
- **Observability:** Metrics are collected; dashboards show system state

3.3 Communication (30%)

Can you explain your system to someone else?

- **Reasoning:** Why did you make each architectural choice?
- **Trade-offs:** What were the alternatives? Why this approach?
- **Troubleshooting:** Can you diagnose and fix problems?
- **Reflection:** What did you learn? What would you change?

Oral Interview (Week 14-15)

Each group will have a 20–30 minute interview where you explain your system and answer questions about your design choices.

4 Week 8: Containerization (Docker)

The Challenge

"How do you package an application so it runs the same way on your laptop, your colleague's machine, and in production?"
Your Assignment 1 server works locally. But when your colleague clones the code and tries to run it, small differences in their system cause failures. Docker solves this by packaging applications and their dependencies into containers that run identically everywhere.

Real-World Problem

The startup needs to:

- Deploy the same application consistently to different machines
- Make it easy for new team members to get started (no environment setup)
- Ensure what works in development works in production
- Move toward containers as the deployment standard

What You'll Learn

- How containers work and why they matter
- Write Dockerfiles that package applications efficiently
- Build and test images locally
- Push images to a registry (Docker Hub) so anyone can use them
- Understand best practices (small images, security, reproducibility)

Your Task

4.1 Core (* Basic): Containerize Two Applications

1. **Nginx Container:**
   - Write a Dockerfile to package a Nginx server
   - Base image: `nginx:latest`
   - Include any custom configuration from Assignment 1
   - Build the image locally: `docker build -t nginx-gsx .`
   - Test locally: `docker run -p 80:80 nginx-gsx`
   - Verify: `curl localhost` returns your site
2. **Simple Application Container:**
   - Write a Dockerfile for a simple Python or Node.js HTTP server
   - The app just needs to respond to HTTP requests (e.g., return "Hello from container")
   - Test it locally
   - You will use and extend this app to demonstrate different situations in later weeks
3. **Push to Docker Hub:**
   - Create a Docker Hub account (free)
   - Log in locally: `docker login`
   - Tag your images: `docker tag nginx-gsx yourusername/nginx-gsx:v1`
   - Push: `docker push yourusername/nginx-gsx:v1`
   - Verify: someone else can pull and run your image

Documentation

- Explain your Dockerfile for each application
- Why did you choose that base image?
- What dependencies does each application have?
- Document the steps to build and run locally

Deliverables

- [ ] Dockerfile for Nginx application
- [ ] Dockerfile for simple application (template or custom)
- [ ] Both images build successfully locally
- [ ] Both images pushed to Docker Hub
- [ ] Verification: pull from Docker Hub and run on different machine
- [ ] Documentation: Dockerfiles explained, design choices justified
- [ ] Git repo with Dockerfiles and documentation

4.2 Intermediate (**): Optimize Your Dockerfiles

If you finish the core deliverables early, try:

- **Multistage Builds:** Reduce image size by separating build and runtime stages
- **Layer Optimization:** Order Dockerfile commands to minimize rebuilds
- **Security:** Run as non-root user; use minimal base images (alpine)
- Document your optimizations: what's the original size vs. optimized?

4.3 Advanced (***): Security Hardening

- Research container security vulnerabilities and rootless containers alternatives
- Scan your images for vulnerabilities: `docker scan yourusername/nginx-gsx`
- Implement security best practices (read-only filesystem, non-root user, minimal capabilities)
- Document security considerations

What You Should Understand

By the end of Week 8, you should be able to answer:

- What is a container? How is it different from a virtual machine?
- What does a Dockerfile do? Why is each line important?
- Why would you use multistage builds?
- What makes a container image "good"? (small, fast, secure, reproducible)
- What are container registries for?

Common Mistakes to Avoid

- **Large base images:** Alpine Linux is small; full Ubuntu is heavy
- **Copying unnecessary files:** Use .dockerignore to exclude build artifacts, git, etc.
- **Running as root:** Create a non-root user for security
- **Not testing:** Always run and test your image locally before pushing
- **Hardcoded values:** Use environment variables for configuration

5 Week 9: Multi-Container Orchestration (Docker Compose)

The Challenge

"How do you run and connect multiple containers that work together?"
Your Nginx container is great, but it needs to talk to other services: a database, a logging agent, a monitoring tool. Running them manually is tedious and error-prone. Docker Compose lets you define an entire multi-container application in a single file.

Real-World Problem

Applications rarely run in isolation. They need:

- Service A to communicate with Service B
- Data to persist when containers restart
- Configuration to be centralized (not baked into images)
- Quick startup of the entire stack (not running commands manually)

What You'll Learn

- Write docker-compose.yml files (YAML format)
- Define multiple services that communicate
- Configure networking, volumes, and environment variables
- Understand what Compose is good for (development) and where it's limited (production)
- Test a multi-container application locally

Your Task

5.1 Core (* Basic): Create a docker-compose.yml

1. **Create the file structure:**
   - Create a directory: `docker-compose/`
   - Add: `docker-compose.yml`
   - Place your Dockerfiles and any required config files
2. **Define services:**
   - **Service 1 (Nginx):** Use the image from Week 8
   - **Service 2 (Backend):** Use the simple app from Week 8
   - **Service 3 (Optional):** Add a third service (e.g., a simple database, logging collector)
3. **Configure networking:**
   - Services should be able to reach each other by service name
   - Example: Nginx can reach backend at `http://backend:3000`
4. **Configure data persistence:**
   - Define at least one volume for persistent data
   - Example: a `/data` directory that survives container restarts
5. **Configuration management:**
   - Use environment variables for configuration (not hardcoded values)

- Create a `.env` file with example values
- Add `.env` to `.gitignore` (don't commit secrets)

6. **Test the stack:**
   - `docker-compose up`
   - Verify all services start and are healthy
   - Test service communication (e.g., curl from one container to another)
   - `docker-compose down`
   - Verify: are volumes still there? (data persists)

Documentation

- Architecture diagram: show the services and how they connect
- Explain each service: what does it do? Why is it needed?
- Document the volumes: what data persists? Why?
- Explain configuration: where do settings come from?

Deliverables

- [ ] docker-compose.yml with 2–3 services defined
- [ ] All services start: `docker-compose up` works
- [ ] Services communicate (test with curl/netcat)
- [ ] Volumes configured and tested (data persists)
- [ ] Environment variables used (not hardcoded config)
- [ ] .env.example file provided (template for configuration)
- [ ] Architecture diagram and explanation
- [ ] Git repo with compose file and documentation

5.2 Intermediate (**): Add Health Checks and Dependencies

- Add health checks to services (`healthcheck` directive)
- Use `depends_on` to specify service startup order
- Set restart policies (e.g., `restart: always`)
- Document why these are important

5.3 Advanced (***): Custom Networks and Logging

- Create custom Docker networks (not default)
- Configure logging drivers (e.g., json-file with size limits)
- Set up service-specific resource limits (memory, CPU)

What You Should Understand

By the end of Week 9, you should be able to answer:

- What does `docker-compose.yml` define?
- How do services in Compose communicate?
- Why do you need volumes? When do you use them?
- How do you manage secrets (API keys, passwords)?
- When is Compose appropriate? When would you use Kubernetes instead?

Common Mistakes to Avoid

- **Hardcoded configuration:** Use env vars, not environment variables in Dockerfiles
- **No volumes:** Services restart, but data disappears—use volumes
- **Wrong service names:** Double-check service names in connection strings
- **Secrets in Git:** Never commit `.env` files with real passwords
- **No logging:** Services fail silently—use `docker-compose logs`

6 Week 10: Orchestration (Kubernetes)

The Challenge

"How do you run containers at scale with automatic recovery and rolling updates?" Docker Compose is perfect for development on your laptop. But production needs more: automatic scaling, failure recovery, rolling updates without downtime. Kubernetes does this—but it's complex. This week, you'll learn why it exists and how to use it.

Real-World Problem

As GreenDevCorp grows:

- Traffic spikes: need to run 5 copies of the web service instead of 1
- A container crashes: how do you restart it automatically?
- You need to deploy a new version: how do you do it without downtime?
- You have 10 servers: how do you manage containers across all of them?

What You'll Learn

- Kubernetes architecture at a high level (pods, deployments, services)
- Write Kubernetes manifest files (YAML)
- Deploy, scale, and manage applications
- Understand service discovery (how containers find each other)
- Know when Kubernetes is overkill and when it's essential

Your Task

6.1 Core (* Basic): Deploy to Kubernetes (Minikube)

1. **Set up Minikube:**
   - Install Minikube (local Kubernetes)
   - Start a cluster: `minikube start`
   - Verify: `kubectl cluster-info`
2. **Write Kubernetes manifests:**
   - Create a directory: `kubernetes/`
   - Write a Deployment manifest for your Nginx service
   - Write a Deployment manifest for your backend service
   - Write Service manifests to expose services
   - Write ConfigMap manifest for configuration
3. **Deploy to Kubernetes:**
   - `kubectl apply -f kubernetes/`
   - Verify pods are running: `kubectl get pods`
   - Check services: `kubectl get services`
4. **Test service communication:**
   - Exec into a pod: `kubectl exec -it <pod-name> -- bash`

- Test connectivity: `curl http://backend:3000`
- Verify services can reach each other by service name

5. **Test scaling:**
   - Scale up: `kubectl scale deployment nginx --replicas=3`
   - Watch pods: `kubectl get pods --watch`
   - Scale down: `kubectl scale deployment nginx --replicas=1`
   - Observe: new pods start, old ones terminate
6. **Test resilience:**
   - Kill a pod: `kubectl delete pod <pod-name>`
   - Observe: Kubernetes automatically starts a new one
   - Check logs: `kubectl logs <pod-name>`

Documentation

- Explain each Kubernetes resource (Deployment, Service, ConfigMap)
- Why do you need each one?
- How do pods communicate? How do external clients reach services?
- Document scaling behavior

Deliverables

- [ ] Minikube cluster running locally
- [ ] Kubernetes manifest files (YAML) for all services
- [ ] All deployments successful: `kubectl get pods` shows running pods
- [ ] Services defined and tested: `kubectl get services`
- [ ] Inter-service communication works
- [ ] Scaling tested: replicas increase/decrease
- [ ] Resilience tested: killed pods are restarted
- [ ] ConfigMaps used for configuration
- [ ] Git repo with manifest files and documentation

6.2 Intermediate (**): Add Resource Limits and Probes

- Define resource requests and limits (CPU, memory)
- Add readiness and liveness probes for health checks
- Document why these matter for production

6.3 Advanced (***): Persistent Storage

- Use PersistentVolume and PersistentVolumeClaim for data
- Understand StatefulSets (for databases or stateful apps)
- Document how data survives pod restarts

What You Should Understand

By the end of Week 10, you should be able to answer:

- What's a pod? How is it different from a container?
- What's a Deployment? Why would you use it instead of running a pod directly?
- How do Services work? Why do you need them for networking?
- What happens when you scale a deployment?
- How does Kubernetes recover from failures?
- When is Kubernetes worth the complexity? When is it overkill?

Common Mistakes to Avoid

- **Wrong selectors:** Service selectors must match pod labels
- **No resource limits:** Pods can consume all available resources
- **No health checks:** Failed pods aren't detected automatically
- **Hardcoded config:** Use ConfigMaps, not environment variables in manifests
- **Assuming persistence:** Data in pods is ephemeral; use volumes

7 Week 11: Infrastructure as Code & CI/CD

The Challenge

"How do you make infrastructure reproducible, version-controlled, and automatically deployable?"
Writing Kubernetes manifests by hand is tedious and error-prone. Infrastructure as Code means writing code that creates your infrastructure. CI/CD means your code is automatically tested and deployed. Together, they form the foundation of modern operations.

Real-World Problem

Currently:

- Infrastructure exists only on your machines
- If something breaks, you manually reapply manifests (from memory)
- New team members don't know how to reproduce the environment
- Deployments are manual and easy to mess up
- No audit trail: who changed what when?

What You'll Learn

- Write Infrastructure as Code (Terraform or Ansible)
- Version-control your infrastructure (it's code!)
- Set up a CI/CD pipeline (GitHub Actions)
- Understand declarative vs. procedural approaches
- Know when each tool is appropriate

Your Task

7.1 Core (* Basic): Write IaC and Deploy Automatically

Choose one: Terraform (declarative) or Ansible (procedural)

Option A: Terraform (Recommended)

1. **Install Terraform:** Download and verify installation
2. **Write Terraform code:**
   - Create a directory: `terraform/`
   - Create `main.tf`: Define Kubernetes resources
   - Create `variables.tf`: Parameterize your deployment
   - Create `outputs.tf`: Show useful information after deployment
   - Organize resources logically (one service per file is fine)
3. **Test Terraform:**
   - `terraform init`: Initialize Terraform
   - `terraform plan`: Show what would be created
   - `terraform apply`: Actually create resources
   - `terraform destroy`: Clean up

4. **Version control your IaC:**
   - Add Terraform files to Git
   - Push to GitHub
   - Document: how to deploy from your code?

Option B: Ansible (Alternative)

1. **Install Ansible:** `pip install ansible`
2. **Write Ansible playbooks:**
   - Create `playbooks/deploy.yml`
   - Define tasks to create Kubernetes resources (using `kubernetes.core.k8s` module)
   - Use variables for configuration
3. **Test playbooks:**
   - `ansible-playbook playbooks/deploy.yml`
   - Verify resources are created
   - Test idempotence: run again, see no changes
4. **Version control:**
   - Add playbooks to Git
   - Document how to deploy

Set Up CI/CD Pipeline

Important constraint (read carefully) GitHub Actions runs on GitHub-hosted runners and **cannot access your laptop's local Minikube cluster**. Therefore, we will split into:

- **CI in GitHub Actions:** build/rebuild container images and validate your IaC configuration.
- **Local CD to Minikube:** you run Terraform/Ansible locally to deploy/update your Minikube cluster.

Your Terraform/Ansible should **use the container images you created in previous weeks** and serve as a **replacement for writing many Kubernetes manifests by hand**. The CI pipeline should ensure that any changes to your code (application, Dockerfile, IaC) are validated and produce deployable artifacts (container images). You will then manually deploy these artifacts to your local Minikube cluster.

1. **Create GitHub Actions workflow (CI only):**
   - Create `.github/workflows/ci.yml`
   - Trigger on **pushes to main**
   - Build (and rebuild) your Docker images, tag them (e.g., with commit SHA), and push them to a registry (Docker Hub or GHCR)
   - Validate your IaC without touching Minikube:
     - **Terraform:** `terraform fmt -check`, `terraform init -backend=false`, `terraform validate`
     - **Ansible:** syntax/lint checks (e.g., `ansible-playbook --syntax-check` and/or `ansible-lint` if you choose to use it)
   - **Do not** run `terraform apply` / `ansible-playbook deploy.yml` in GitHub Actions against Minikube
2. **Local CD to Minikube (deployment step):**
   - Make a change (application code, Dockerfile, or IaC)
   - Push to GitHub and ensure the CI workflow is green
   - On your machine: `minikube start` and deploy using your IaC tool
   - Verify locally: your Minikube cluster is updated and **running the new image tag**

Documentation

- Explain your IaC tool choice (Terraform vs. Ansible)
- Document how to deploy: steps from code to running infrastructure
- Document the local deployment workflow: **CI in GitHub Actions + local CD to Minikube**
- Document how you choose the image tag produced by CI (e.g., commit SHA) and where it is configured in Terraform/Ansible
- Explain variables and outputs
- Describe the CI/CD pipeline: what happens on push? On merge?

Deliverables

- [ ] IaC code (Terraform or Ansible) defining full application stack
- [ ] Code organized logically (not all in one file)
- [ ] Variables used for configuration (not hardcoded)
- [ ] Terraform/Ansible can deploy from scratch: verify with `destroy` then `apply/run`
- [ ] CI workflow configured (.github/workflows/) and passing
- [ ] CI builds/rebuilds images and pushes them to a registry
- [ ] CI validates IaC (no remote apply from GitHub Actions)
- [ ] Tested end-to-end: push changes → CI green → local deploy to Minikube updates the running stack
- [ ] Documentation: tool choice, deployment steps, pipeline explanation
- [ ] Git repo with IaC code and CI/CD configuration

7.2 Intermediate (**): Multiple Environments

- Use variables/tfvars to define dev and staging environments separately
- Deploy both with one codebase
- Document: how do you ensure staging is tested before prod?

7.3 Advanced (***): Advanced CI/CD

- Add container security scanning in CI (fail on high/critical vulnerabilities)
- Generate and upload build artifacts (e.g., SBOM) as workflow artifacts
- Improve build performance with caching (e.g., Docker Buildx cache)
- Add a simple release/tag strategy (e.g., commit SHA tags for CI, and a manual/published "stable" tag)
- Define and test a rollback procedure (e.g., redeploy previous image tag and/or `kubectl rollout undo`)

What You Should Understand

By the end of Week 11, you should be able to answer:

- What's Infrastructure as Code? Why does it matter?
- What's the difference between Terraform (declarative) and Ansible (procedural)?
- Why version-control infrastructure?
- What does a CI/CD pipeline do?
- How do you ensure infrastructure changes are safe?

Common Mistakes to Avoid

- **Hardcoded values:** Use variables for anything that might change
- **No version control:** IaC code should be in Git, not lost somewhere
- **Incomplete IaC:** If you manually edit manifests, IaC is out of sync
- **No testing:** Always run plan before apply; test in non-prod first
- **Secrets in Git:** Never commit API keys or passwords

8 Week 12: Network Design & Identity

The Challenge

"How do you design a network for a growing organization? How do you manage who can access what?"
Your containerized infrastructure works in isolation. But real organizations have security boundaries: development shouldn't access production, external partners should have limited access, and authentication should be centralized. This week is about thinking like a security architect.

Real-World Problem

GreenDevCorp is growing:

- Two offices across countries (they need to connect securely)
- Multiple teams: developers, data analysts, operations (different access needs)
- Development, staging, and production environments (must be isolated)
- Partners and contractors (temporary access)
- Compliance: prove that only authorized people accessed sensitive data

What You'll Learn

- Network design principles (segmentation, defense-in-depth)
- CIDR notation for IP addressing
- Understanding DNS, DHCP, NTP (why they matter, not building them)
- Kubernetes NetworkPolicies for segmentation
- Identity concepts: authentication, authorization, centralized identity

Your Task

8.1 Core (* Basic): Design Network Architecture

1. **Create a network architecture diagram:**
   - Show multiple environments: development, staging, production
   - Show network segments (DMZ, internal, database)
   - Include external connections (internet, partner access)
   - Use text boxes, ASCII art, or a simple drawing tool (check Mermaid, draw.io)
2. **Plan IP addressing:**
   - Use CIDR notation for subnets
   - Example: `10.0.0.0/16` for entire organization
   - Subdivide:
     - `10.0.1.0/24` for development (how many IPs?)
     - `10.0.2.0/24` for staging
     - `10.0.3.0/24` for production
     - `10.0.10.0/24` for external partners
   - Document: why this subdivision? What's in each subnet?
3. **Implement Kubernetes NetworkPolicies:**
   - Create NetworkPolicy manifests to enforce segmentation

- Example: production pods cannot access development pods
- Example: external pods cannot reach internal database
- Deploy policies and test them

4. **Document security boundaries:**
   - What traffic is allowed between networks?
   - What traffic is blocked? Why?
   - How do you prevent accidental misconfiguration?

Research: Core Services

1. **DNS (Domain Name System):**
   - Research: what is DNS? What problem does it solve?
   - Why does an organization need DNS?
   - How does DNS work (high-level)?
   - Write 1–2 paragraphs explaining DNS to a non-technical person
2. **DHCP (Dynamic Host Configuration Protocol):**
   - Research: what is DHCP? What problem does it solve?
   - Why would an organization use DHCP?
   - How does DHCP work (high-level)?
   - Write 1–2 paragraphs explaining DHCP
3. **NTP (Network Time Protocol):**
   - Research: what is NTP? Why does time synchronization matter?
   - Why is synchronized time important for security and operations?
   - Write 1–2 paragraphs explaining NTP

Research: Identity Management

1. **Authentication vs. Authorization:**
   - What's the difference?
   - Write 1–2 paragraphs explaining each
2. **Centralized Identity:**
   - Research: LDAP (Lightweight Directory Access Protocol)
   - Research: Active Directory (Microsoft's implementation)
   - Research: SSO (Single Sign-On) and why it matters
   - Write: what problem does centralized identity solve?
   - When would a small company need this? A large company?
3. **Identity strategy for GreenDevCorp:**
   - Given the company size (20+ people), what identity solution would you recommend?
   - Why?
   - What are the trade-offs?
   - Document your recommendation with reasoning

Deliverables

- [ ] Network architecture diagram (text, ASCII art, or drawn)
- [ ] CIDR plan with documented reasoning for each subnet
- [ ] Kubernetes NetworkPolicies enforcing segmentation
- [ ] Evidence: NetworkPolicies deployed and tested
- [ ] Research document: DNS, DHCP, NTP (1–2 paragraphs each)
- [ ] Identity research: LDAP, AD, SSO (explanations)
- [ ] Identity strategy recommendation for GreenDevCorp (with reasoning)
- [ ] Security analysis: what could go wrong? How would you mitigate?
- [ ] Git repo with network diagrams and research documents

8.2 Intermediate (**): Advanced Network Design

- Design VPN connectivity between offices
- Implement network policies with more complex rules (CIDR blocks, ports)
- Document: how would you expose a service to external partners?

8.3 Advanced (***): Identity Implementation (Optional)

- Set up OpenLDAP with 3–5 test users
- Configure a Kubernetes cluster to authenticate against LDAP
- Document: users can authenticate via LDAP to access cluster
- This is optional and only if you have significant time left

What You Should Understand

By the end of Week 12, you should be able to answer:

- Why segment networks? What do you protect by doing this?
- What's a subnet? CIDR notation?
- Why do organizations use DNS, DHCP, NTP?
- What's the difference between authentication and authorization?
- Why might a company use centralized identity?
- How do you ensure secure communication between networks?

Common Mistakes to Avoid

- **Poor segmentation:** Development and production should be completely isolated
- **No default deny:** Start with "deny everything" then allow specific traffic
- **Overly complex policies:** Start simple, refine as needed
- **Assuming security is solved:** Network policies are one layer; other protections needed

9 Week 13: Integration, Observability & Finalization

The Challenge

"Pull it all together. See what's happening. Document everything. Get ready to explain it."

You've built containers, orchestrated them, automated infrastructure, and designed networks. Now integrate everything, add visibility, and prepare to hand it off to someone else (or your future self).

Real-World Problem

Week 13 is consolidation week:

- You need to pull all components together into one coherent system
- You need visibility: metrics, dashboards, alerting
- You need documentation so others can understand and operate your system

What You'll Learn

- Observability: collecting and visualizing system metrics
- Creating dashboards that communicate system health
- Defining what should be monitored and why
- Integration: making sure all components work together
- Documentation: explaining your system clearly

Your Task

This week has four mini-challenges. The priority is to finish the tasks of previous weeks first. Then, complete as many of the following challenges as you can within your available time. Prioritize in this order: Challenge B, Challenge C, Challenge D, Challenge A.

9.1 Challenge A: Observability with Prometheus & Grafana (Optional but Recommended)

Core (* Basic): Deploy Prometheus and Grafana

1. **Deploy Prometheus to Kubernetes:**
   - Add Prometheus manifests to your Kubernetes deployment
   - Or add Prometheus service to your docker-compose.yml
   - Configure scrape targets: your application pods, node metrics
2. **Create a Grafana dashboard:**
   - Deploy Grafana to Kubernetes/Compose
   - Connect to Prometheus datasource
   - Create a dashboard showing:
     - Request rate and latency (from your app)
     - CPU and memory usage (from nodes/pods)
     - Error rate (from logs or metrics)
     - Any other metrics you care about
3. **Test the observability stack:**

- Generate traffic to your application
- Watch metrics update in real-time
- Create a dashboard screenshot showing system health

Intermediate (**): Add Alerting

- Define alert rules: what indicates a problem? (e.g., error rate > 5%, CPU > 80%)
- Configure alerts to send notifications (log files, or email if available)
- Test: trigger conditions and verify alerts fire

9.2 Challenge B: Full Integration Test (Required)

1. **Start from scratch:**
   - Delete all Kubernetes pods/deployments or stop all containers
   - Verify everything is gone
2. **Deploy from your IaC code:**
   - `terraform apply` or run Ansible playbooks
   - Watch all services come up automatically
3. **Verify end-to-end:**
   - Services are running
   - Services communicate with each other
   - External access works (can reach Nginx/load balancer)
   - NetworkPolicies are enforced
   - (If completed Challenge A) Metrics are being collected
4. **Document the process:**
   - How long did deployment take?
   - Any issues? How did you fix them?
   - Screenshots showing all components working

9.3 Challenge C: Documentation (Required)

1. **Architecture diagram:**
   - Show all components: containers, services, networks, load balancers
   - Show data flows: how does traffic move through the system?
   - Use ASCII art, text description, or a drawn diagram
2. **Component documentation:**
   - What does each microservice do?
   - What are its dependencies?
   - How is it deployed? How is it configured?
3. **Operational runbook:**
   - How do you deploy a new version?
   - How do you scale a service?
   - How do you check logs?
   - How do you troubleshoot a failure?

- How do you access the observability dashboard (Challenge A)?

4. **Troubleshooting guide:**
   - Common problems and how to diagnose them
   - Example: "Service X can't reach Service Y. How do I debug?"
   - Example: "Dashboard shows high error rate. What do I check?"
5. **README:**
   - Overview of the project
   - Quick start: how to get the system running?
   - Links to other documentation

9.4 Challenge D: Reflection & Interview Prep (Required)

1. **Reflection essay (individual, not paired):**
   - What was the most challenging aspect of this assignment?
   - What surprised you about modern infrastructure?
   - What would you do differently if you started over?
   - How has your understanding of DevOps and cloud-native systems changed?
   - What do you want to learn more about?
   - Length: 500–1000 words
2. **Prepare for oral interview:**
   - Practice explaining your system out loud
   - Be ready to defend your architectural choices
   - Prepare for questions like:
     - "Why did you choose Kubernetes over Compose?"
     - "A container is crashing. Walk me through debugging."
     - "How would this system handle 10x traffic?"
     - "What would you change given more time?"
   - Note: Both team members should be able to explain the entire system

Deliverables

- [ ] (Optional) Prometheus + Grafana deployed, dashboard created
- [ ] Full integration test: deploy from IaC code, verify everything works
- [ ] Architecture documentation (diagram + explanation)
- [ ] Operational runbook (how to deploy, scale, troubleshoot)
- [ ] Troubleshooting guide (common problems and solutions)
- [ ] README with quick start and links to documentation
- [ ] Individual reflection essay (each student)
- [ ] Prepared for oral interview (both students)
- [ ] Clean Git repo with all code and documentation

What You Should Understand by the End

- What modern cloud-native infrastructure looks like
- Why each technology (Docker, K8s, IaC, monitoring) matters
- How these technologies work together
- How to reason about infrastructure design
- When to use each tool and when it's overkill

10 Grading and Assessment

10.1 Three Pillars

Your work will be evaluated on three dimensions.

1. **Process (40%)**
   - **Code Quality:** Dockerfiles, manifests, IaC code are clean, well-organized, and documented
   - **Version Control:** Git history is clear; commits have meaningful messages
   - **Automation:** Infrastructure is version-controlled and reproducible
   - **Best Practices:** You follow conventions (e.g., no hardcoded values, proper .gitignore)
   - **Collaboration:** Both team members contribute; evidence in Git
2. **Functionality (30%)**
   - **Docker:** Images build, run, and are pushed to registry correctly
   - **Compose:** Services start, communicate, and data persists
   - **Kubernetes:** Deployments run, services route traffic, scaling works
   - **IaC:** Code can deploy from scratch; is idempotent
   - **Network:** Policies are enforced; segmentation works
   - **Observability:** Metrics collected (if completed); dashboards show system state
3. **Communication (30%)**
   - **Explanation:** You can explain each component and why it exists
   - **Reasoning:** You can defend architectural choices and trade-offs
   - **Troubleshooting:** You can diagnose problems using logs and metrics
   - **Reflection:** You understand what you learned and what challenged you

10.2 Difficulty Levels and Expectations

Each week has difficulty levels. Your grade reflects the level you attempt:

- *** Basic:** All core deliverables working. Grade: 5-7 range.
- **** Intermediate:** Core + intermediate challenges. Grade: 8-9 range.
- ***** Advanced:** Core + intermediate + advanced exploration. Grade: 10 (if done well).

Important: It's better to complete basic-level deliverables thoroughly than to rush through advanced challenges. **Quality over quantity.**

10.3 Oral Interview

Each group will have a 20–30 minute interview in Week 14 or 15. You'll be asked to:

- Demonstrate your system running
- Explain your architecture
- Answer questions about your design choices
- Troubleshoot a problem scenario
- Discuss what you learned

Both team members should be present and prepared to answer questions about the entire system.

11 Guidelines and Expectations

11.1 Collaboration

- **Equal contribution:** Both team members should contribute meaningfully every week
- **Rotating roles:** One person leads research this week, the other leads implementation
- **Regular communication:** Discuss architectural decisions before implementing
- **Git evidence:** Commits from both should be visible

11.2 Problem-Solving Approach

- **Start with research:** Understand what you're trying to do before building
- **Make incremental progress:** Small steps with testing, not big leaps
- **Document as you go:** Don't leave documentation until the end
- **Test thoroughly:** Verify each component works before moving on
- **Iterate:** First attempt rarely perfect; refine based on what you learn

11.3 Version Control Best Practices

- Commit frequently (multiple commits per week)
- Write clear commit messages:
  - Good: "Add Kubernetes deployment for backend service; configure health checks"
  - Poor: "fix stuff"
- Use .gitignore to exclude:
  - `.env` files with secrets
  - Build artifacts (`dist/`, `build/`, `__pycache__/`)
  - Large files (logs, backups, Docker images)
- Never commit passwords, API keys, or credentials

11.4 Safety and Best Practices

- **Test in non-production first:** Develop on your laptop, test in staging before prod
- **Back up your work:** Commit to Git frequently
- **Don't hardcode secrets:** Use environment variables or secrets management
- **Follow least privilege:** Give containers and users only what they need
- **Document as you go:** Don't delay writing documentation

11.5 Getting Help

- **Lab sessions:** Bring specific questions; professors are there to unblock you
- **Documentation:** Read man pages and official docs before asking
- **Collaborate with peers:** Other groups are learning the same things
- **Be specific:** "My Kubernetes deployment isn't starting" is vague. "Deployment shows ImagePullBackOff error; here's my manifest" is helpful

12 Resources and References

12.1 Recommended Reading

- *UNIX and Linux System Administration Handbook*, 5th Edition (Chapters on containers, Kubernetes)
- *The Practice of Cloud System Administration*, Volume 2 (Chapters on containers, IaC, cloud operations)
- Official Docker documentation: `https://docs.docker.com/`
- Official Kubernetes documentation: `https://kubernetes.io/docs/`
- Terraform documentation: `https://www.terraform.io/docs/`
- Ansible documentation: `https://docs.ansible.com/`
- Prometheus documentation: `https://prometheus.io/docs/`
- Grafana documentation: `https://grafana.com/docs/`
- Google the rest!

12.2 Tools and Accounts You'll Need

- **Docker Desktop:** Install on your laptop (includes Docker and Docker Compose)
- **Docker Hub:** Free account for image registry
- **Minikube:** Free local Kubernetes
- **GitHub:** Free account for version control and CI/CD (GitHub Actions)
- **Git:** Version control tool (install locally)
- **Text editor:** VSCode, Vim, or your favorite
- **kubectl:** Command-line tool for Kubernetes (comes with Minikube)
- **Terraform:** Infrastructure as Code (open-source, free)

All of these are free.

12.3 Key Commands You'll Use

- **Docker:** `docker build`, `docker run`, `docker push`
- **Docker Compose:** `docker-compose up`, `docker-compose down`
- **Kubernetes:** `kubectl apply`, `kubectl get`, `kubectl logs`
- **Git:** `git add`, `git commit`, `git push`
- **Terraform:** `terraform init`, `terraform plan`, `terraform apply`, `terraform destroy`

13 Final Thoughts

This assignment mirrors real infrastructure work. You'll face ambiguous problems, discover solutions, make trade-off decisions, and document your work. You'll collaborate with teammates, use industry-standard tools, and build something that actually works.

The most important skill you'll develop is not memorizing commands or syntax. It's learning to **think like an infrastructure engineer:**

- "What problem are we solving?"
- "What tools are available?"
- "What are the trade-offs of each approach?"
- "How do we make this reliable and observable?"
- "How do we document so others can maintain it?"

You'll encounter technologies and concepts you don't fully understand. That's intentional. The goal is not mastery—it's fluency. Know what each tool does, why it exists, and when to use it. Deep expertise comes later.

Good luck, and remember: the best infrastructure engineers are never done learning.